

Code Analysis

Names of the programs: qwenSource.c and geminiSource.c

Accessed by: qwenSource through Odysseus, a self-hosted AI workspace for chats and agents, and gemini through gemini desktop app.

Prompt used for both GenAI:

Write a C program that picks a secret number between 1 and 10. The user gets up to 3 tries to guess the number.

The requirements are as follows:

- Use a fixed secret number (use 7)
- Use printf to prompt the user and scanf to read their guess
- use a while or for loop to let the user guess up to 3 times
- use if-else statements to check if the guess is too high, too low, or correct
- stop the loop immediately if the user guesses the correct number
- print a final win or lose message

Generated code can be seen in further pages.

In terms of correctness, the number guessing game created using qwen3:1.7b and gemini pro run fine under an assumption that the user will use integers as inputs. Inputs other than integers will crash the program. And no meaningful errors are made by both.

Time complexity wise, for loop (gemini Pro) is typically faster than while loops (qwen3:1.7b)

Space complexity: Qwen3:1.7b uses only 3 meanwhile Gemini Pro uses 4

Maintainability: Gemini Pro comments are minimal, but better than Qwen3:1.7b which had none. A code walkthrough through both would suggest that it would crash if user inputs anything but an integer.

I selected the code generated by Qwen3:1.7b because it had more potential improvements.

The improvements:

- Correctness: inputs other than integer will ask user again to try again and an error message is provided.
- Time complexity: changing while loop into for loop.
- Space complexity: already using 3 constants instead of 4.
- Maintainability: add line by line comments. A code walkthrough suggests that the string/special character input edge case is now handled.

Qwen3:1.7b

```
#include <stdio.h>

int main() {

    int secret = 7

    int guess, attempts = 0;


    while (attempts < 3) {
        printf("Enter your guess (1-10): ");
        scanf("%d", &guess);

        if (guess == secret) {
            printf("Win! You guessed the correct number.\n");
            break;
        } else if (guess > secret) {
            printf("Too high! Try again.\n");
        } else {
            printf("Too low! Try again.\n");
        }

        attempts++;
    }

    if (attempts == 3) {
        printf("Game Over! You failed to guess the correct number.\n");
    }

    return 0;
}
```

Gemini Pro

```
#include <stdio.h>

int main() {

    // Fixed secret number as required

    int secret_number = 7;

    int guess;

    int max_attempts = 3;

    int won = 0;


    printf("Welcome to the Guessing Game!\n");

    printf("I have picked a secret number between 1 and 10.\n");

    printf("You have %d attempts to guess it.\n\n", max_attempts);


    // Loop lets the user guess up to 3 times
    for (int attempt = 1; attempt <= max_attempts; attempt++) {

        printf("Attempt %d: Enter your guess: ", attempt);


        // Read the user's guess

        scanf("%d", &guess);


        // Check if the guess is correct, too high, or too low
        if (guess == secret_number) {

            won = 1;

            break; // Stop the loop immediately on a correct guess

        } else if (guess > secret_number) {

            printf("Too high!\n");

        } else {

            printf("Too low!\n");

        }

    }

}
```

```
// Final win or lose message
printf("\n");
if (won) {
    printf("Congratulations, you win! You guessed the secret number.\n");
} else {
    printf("Game over. You lose! The secret number was %d.\n", secret_number);
}

return 0;
}
```

Final Qwen3:1.7b code:

// all code is generated by qwen3:1.7b

```
#include <stdio.h>          // import standard input output library

int main() {                // main function takes no params

    int secret = 7;          // secret is 7

    int guess = 0;           // guess is 0

    int max_attempts = 3;    // max_attempts set to 3

    for(int attempt = 0; attempt < max_attempts; attempt++) {      // repetition while statement that
test if attempts is lesser than 0

        printf("Enter your guess (1-10): ");    // ask user to enter the guess

        // generated by Gemini Pro

        if (scanf("%d", &guess) != 1) {        // scan the user number

            printf("Error: That is not a valid integer. Please try again.\n"); // output error to user

            // <label> Clear the invalid characters from the input buffer

            int c;

            while ((c = getchar()) != '\n' && c != EOF) {

                // <label> Do nothing, just consume the characters

            }

            // <label> Subtract 1 from attempt so the user doesn't lose a turn for a typo

            attempt--;

            continue;

        }

        if (guess == secret) {                // if guess equals to secret

            printf("Win! You guessed the correct number.\n");    // then print win

            break;                // break out of while loop
```

```
} else if (guess > secret) {           // else if guess is greater than secret
    printf("Too high! Try again.\n");   // then print Too high! Try again
} else {                               // otherwise
    printf("Too low! Try again.\n");    // print Too low! Try again
}
}

if (guess != secret) {                 // checks final guess against secret if they are not
equal
    printf("Game Over! You failed to guess the correct number.\n"); // then print game over
message
}
return 0;
}
```